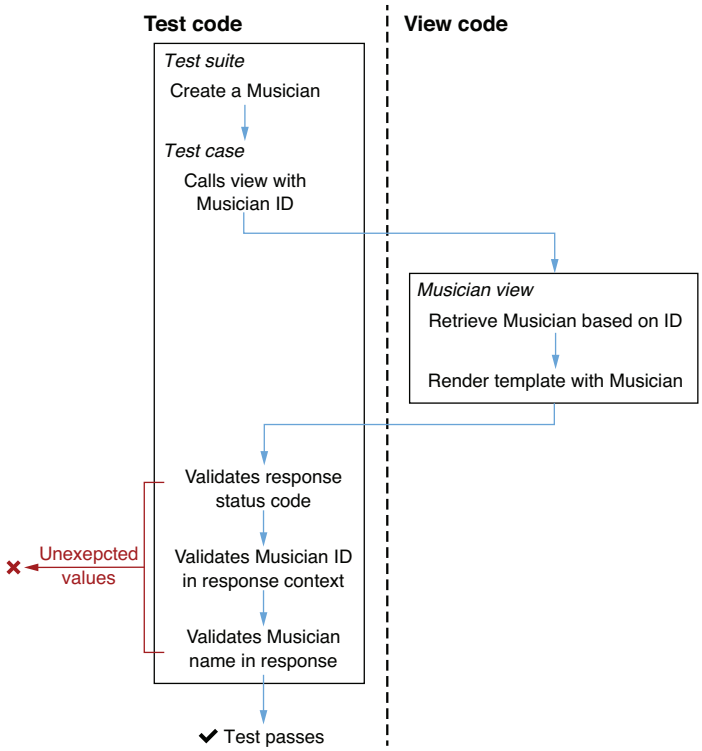


Figure 8.3 Testing the `musician()` view

like with the `credits()` view, you visit the musician view using the `.client.get()` method. There are two schools of thought here: you can look the URL up, or you can hardcode it. Looking the URL up makes your test more bulletproof. Hardcoding it catches a problem if you've accidentally moved the URL route. I tend to hard code my URLs in my tests, but that means if I do want to move a URL, I have to make changes to all my tests.

The musician view takes a `Musician` ID, which is sent in through the URL. The view looks up the corresponding `Musician` object and then renders a template, displaying the musician information. To test this view, you want to validate the HTTP response code, check the `Musician` in the view is the correct one, and possibly verify some of the HTML. You already saw how to check the `200` status code in the `credits()` view test. The other two things get checked by examining the response object returned by `.client.get()`. It contains a couple useful attributes: `.context` is the context object passed to the template rendering engine, and `.content` contains the actual rendered response. You can assert that the `Musician` object in the view's context is the same as the one created by the test, and you can also check for values rendered in the HTML.

NOTE `response.content` is stored as a binary object, so you need to convert it to a string to do a comparison.